Exercise - Synthesis

This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

Overview

As modern electronics evolve, the complexity of digital circuits has increased dramatically. Designers can no longer afford to manually create circuits at the gate level for every function. Instead, as you learned in the VLSI-1 lecture, high-level design abstractions are essential to describe circuits efficiently. In this context, we use Hardware Description Languages (HDLs), such as Verilog, SystemVerilog, or VHDL. With HDLs, designers specify how a system should behave (e.g., arithmetic operations, data storage) without focusing on the low-level implementation details of logic gates. This is where synthesis becomes essential.

Synthesis is the process that bridges the gap between these high-level hardware descriptions and their implementation in terms of gates and transistors. Synthesis tools automate the conversion of high-level designs into optimized gate-level netlists, which can later be implemented in hardware.

Furthermore, synthesis plays a key role in optimizing designs for performance, area, and power. Modern synthesis tools incorporate algorithms that balance trade-offs among these factors, guided by the designer's constraints. For example, if low power consumption is a priority for a mobile device, the synthesis tool can generate a design that reduces power usage. Conversely, if high performance is required, such as in high-speed computing systems, the tool can focus on meeting timing requirements while managing area constraints.

Goals of this exercise

- Understand how the synthesis process works using Yosys
- Learn how to apply and use constraints in synthesis
- Evaluate the quality of synthesis results
- Get familiar with netlists and standard cells used in circuit design

Basic Synthesis Steps

Logic synthesis is the process of mapping a design from a high-level description (in our case, a SystemVerilog implementation) to a netlist of logic gates belonging to a given logic library. As seen in the lecture, EDA synthesis tools like Yosys take as input the HDL design description, a set of constraints (e.g. the target clock period), and a set of logic libraries from the target technology and output a gate-level netlist consisting of only gates from the provided technology logic library and wires to connect them, as shown in Figure 1.

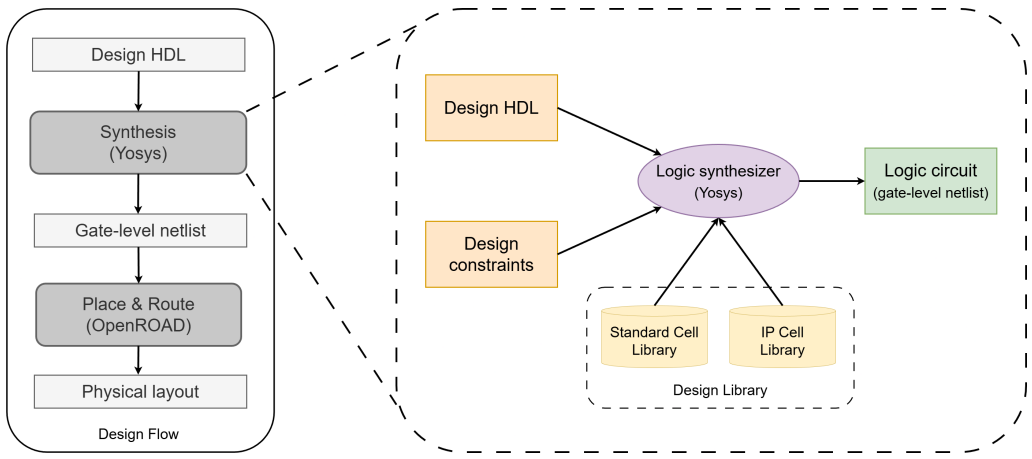

Figure 1: Synthesis flow

In this class we use the open-source Yosys tool for synthesis. In Yosys, synthesis is a multi-stage process, typically consisting of the following four main steps:

- **Front-end Preprocessing**: This is the initial stage where Yosys reads the RTL code. It parses the code and generates an internal representation of the design, which is stored as an abstract syntax tree (AST). This internal representation forms the basis for further transformations.

- **Elaboration**: During this phase, Yosys expands all module instantiations, resolves parameters, and, if necessary, flattens the hierarchy. This ensures the design is fully prepared for synthesis, producing a netlist that includes logic gates and higher-level constructs such as flip-flops and multiplexers. The elaborated netlist represents a fully resolved version of the original RTL, ready for further processing.
- **Coarse-grain Synthesis**: Yosys translates the RTL into a gate-level netlist. This involves logic synthesis, where combinational and sequential logic is mapped to basic gates like AND, OR, XOR, and registers (flip-flops). Yosys is also capable of optimizing the design for logic minimization.
- **Technology Mapping**: In this step, Yosys uses ABC (http://www.eecs.berkeley.edu/~alanmi/abc/) to map the optimized netlist to a specific target technology, such as an FPGA or ASIC standard cell library. If targeting FPGAs, the design is mapped to LUTs, flip-flops, and other resources like DSP blocks and block RAMs. For ASICs, the design is mapped to the standard cells (e.g., NAND, NOR, DFF). ABC uses the provided technology library and timing constraints to optimize the design.

In this exercise, we will guide you through using Yosys to synthesize an example design, **Croc**.

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex04
sh > cd ex04
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex04/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex04
```

Synthesizing Croc

In this section, we will guide you through the detailed synthesis flow. Let us start with some guidelines on how to run Yosys.

> ***Hint:*** *As synthesis, and other backend flows, are often iterative, it's a good practice to keep track of the commands you execute in Yosys. This helps save time during iterations and streamlines the process. To assist you with this, we provide a script template at* `croc/yosys/scripts/yosys_flow.tcl`*. During this exercise you will be asked to fill in this template; this will allow you to document your workflow and easily re-run your commands without starting from scratch each time.*

Note: you do not have to run the following commands now, these are to show some useful basic commands to start the synthesis tool, run scripts, etc. The commands you are required to execute for this exercise are located within the student task boxes throughout the exercise.

You can use the following commands to start the Yosys interactive shell. When working on Croc, it is good practice to start each tool from within the directory named accordingly (i.e. `croc/yosys` for Yosys).

```
sh (yosys) > oseda -2025.01 yosys -C
```

The `-C` (short for `--tcl-interactive`) command line option tells Yosys to start an interactive TCL command line shell. TCL is the default scripting languange in most IC design EDA tools, so we will always use it with this option. Note that this implies that Yosys-specific commands will require the *yosys* prefix when executed from this shell, as you will see later. Finally, the `-c, --tcl-scriptfile <tcl_scriptfile>` option instructs Yosys to run a TCL script in non-interactive mode. This can be helpful for scripting if you need to run multiple TCL scripts.

To run the script in the Yosys interactive shell:

```
yosys > source <your/script/path>
```

In this exercise you will be asked to fill some commands into a synthesis script, which you can execute as:

```
yosys > source scripts/yosys_flow.tcl
```

To re-run the flow, use the following command:

```
     yosys > exit                            # exit from the current synthesis flow
sh (yosys) > rm -r tmp/ out/* reports/* *.log        # remove generated files
sh (yosys) > oseda -2025.01 yosys -C
     yosys > source scripts/yosys_flow.tcl
```

Alternatively, you can also run the flow in non-interactive mode:

```
sh (yosys) > oseda -2025.01 yosys -C scripts/yosys_flow.tcl
```

Load Technology Libraries

In this section, we will load the target technology library, which is essential for the synthesis flow. A technology library defines the characteristics of standard cells, including their electrical properties, timing, and power models. These are critical for ensuring that the design meets timing, area, and power constraints during synthesis and further backend flow. For this exercise, we will use the open-sourced **IHP-SG13G2 (https://github.com/IHP-GmbH/IHP-Open-PDK) technology library**, a 130nm CMOS process.

The IHP-SG13G2 library is composed of three components:

- **Standard cells library**: Contains logic gates, flip-flops, and other basic elements used to build the core of the digital design.
- **SRAM library**: Provides memory blocks such as SRAM, commonly used for data storage within the design.
- **I/O library**: Contains input/output pads for interfacing the design with external components.

You can directly inspect the technology library files since they are stored in the human-readable **.lib** format by navigating to the technology directory. For ETH users, the **.lib** files will be located under `croc/technology/lib` (note that this directory will only be present after you installed the target technology files through Cockpit, as explained in student task 2 and in the Exercise setup page). For non-ETH users, you can find the equivalent files under `croc/ihp13/pdk/ihp-sg13g2/libs.ref`.

Library file names follow this format:

```
<process>_<library_type>_<operating corner>_<voltage>_<temperature>.lib
```

For example, the file name

```
sg13g2_stdcell_typ_1p20V_25C.lib
```

can be broken down as follows:

- **sg13g2**: The process technology used.
- **stdcell**: Indicates that this is a standard cell library.
- **typ**: Denotes the typical operating corner.
- **1p20V**: The nominal voltage of 1.2 Volts.
- **25C**: The characterization temperature of 25°C.

Let's open the file `technology/lib/sg13g2_stdcell_typ_1p20V_25C.lib` and have a look what is inside. First, the file specifies the default units, default parameters, wire resistance and capacitance models, power models etc. Then the library specify the characteristics for each standard cell in the technology. Lets take buffer `sg13g2_buf_1` as an example. The library first specifies the area and leakage power of the cell at different conditions. Then, it describe the characteristics of the output pin X. It first describe its function as: `function : "A";`. This means that the X will output the same value as A, which is exactly what a buffer is doing. This information is needed by the synthesis tool to correctly select the standard cells to implemente the logics. Next, the library specify the minimum and maximum capacitance of the load the pin X can drive. After this, you will see several tables describing the timing and power behavior of the cell. The sysnthesis tool will use these information to generate a netlist that implement the logic and satisfy the constraints.

In this exercise, we will use the typical corner at 1.2V and 25°C, represented by files named with the suffix `typ_1p20V_25C`. This corner represents nominal conditions, which are commonly used for initial synthesis and timing analysis. Later in the design process, you may explore other corners (such as slow or fast) to conduct comprehensive timing verification under worst-case or best-case conditions.

Once you've identified the technology libraries used in your design and located their paths, the next step is to load these libraries into Yosys. This allows the synthesis tool to utilize the standard cells, SRAMs, and I/O pads for the mapping and synthesis of your design.

Student Task 2: Read Technology Libraries into Yosys

To begin, install the technology files for the IHP130nm technology through Cockpit (only for ETH users).

```
sh (ex04) > cd croc
sh (croc) > icdesign ihp13 -update all -nogui
```

Locate the files corresponding to the **Standard cells library**, **SRAM library**, and **I/O library** in the technology directories mentioned above. Make sure these files are accessible before proceeding with the synthesis flow. If unsure, ask a TA for help.

Next, open a Yosys interactive shell from a TCL shell using the following command. **Make sure you do this in yosys folder.** This is assumed by the script:

```
sh (croc)  > cd yosys
sh (yosys) > oseda -2025.01 yosys -C
```

This will launch Yosys within your current terminal tab. To read the libraries into Yosys, use the following command:

```
yosys > yosys read_liberty -lib <path_to_library_file>
```

- **Load the technology library files into Yosys.** Make sure to load all the library files for the typical corner at 1.20V and 25°C.
- Add these commands to the corresponding section in the `yosys_flow.tcl` script.

Load the Design

The next step in the synthesis flow is to load the design files into Yosys. Yosys is capaible of reading and processing Verilog-2005 code, and can process SystemVerilog designs through [yosys-slang (https://github.com/povik/yosys-slang)](https://github.com/povik/yosys-slang) plugin. To enable the SystemVerilog frontend plugin, we will use the following command:

```
yosys > yosys plugin -i slang.so
```

You can use the help `yosys help read_slang` command in Yosys to learn more about the available options and how to configure the command (this only works if the plugin has been enabled).

After the plugin has been enabled, we can use the `read_slang` command, which allows us to load SystemVerilog files. The general syntax of the command is as follows:

```
yosys > yosys read_slang --top <top_design> -f <file_list> [--options]
```

This command specifies the **top-level design module** and provides **a list of the source files** that make up the design. The `--top` option defines the name of the top module, while the `-F` option is used to specify the file list that includes all necessary design files. Note that we use the `-f` option instead of the `-F` one to make sure the source files relative paths are evaluated starting from the current directory (i.e. the `yosys` directory in our case). The name of the top module is `croc_chip`.

After the design has been parsed through Slang, it is possible to print information about the loaded design with the `stat` command. The `write_verilog` command is used to save the current design as a Verilog netlist. You can check the [documentation (https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd_ref.html)](https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd_ref.html) for more information about these commands. To get you started, here are some examples on how to use these commands:

```
# print the report in the Yosys interactive shell
yosys > yosys stat
# export the report into a file named 'croc.rpt'
yosys > yosys tee -q -o "reports/croc.rpt" stat
# export the netlist
yosys > yosys write_verilog "out/croc.v"
```

Student Task 3: Load the design into Yosys

- 3.1 Enable the Slang SystemVerilog frontend plugin.
- 3.2 Load the Croc chip design into Yosys using the `read_slang` command. Be sure to specify the correct top-level design and provide the appropriate file list. **Note**: Slang is quite pedantic in parsing SystemVerilog files in its default configuration. To avoid errors due to harmless sub-optimal coding styles, use the `--compat-mode` `--allow-use-before-declare` and `--ignore-unknown-modules` options.
- Report statistics and export the design
  - Generate an initial design report by using the `stat` command, and save it in a file (e.g. `reports/croc_parsed.rpt`). Use the *-width* option to explicitly state the bit width of each logical block recognized by Yosys in your design.
  - Generate a Verilog netlist from the parsed design, and save it in a file called `croc.analysys.v`.
  - Take a look at the generated report and netlist. How many levels of hierarchy are included in the design?

You may find that the generated netlist is very difficult to interpret, because all the modules have been merged into a single huge block, named `croc_chip`. This approach is not ideal because it eliminates the inherent modularity of the design, making it much harder to isolate and debug individual components. Without clear module boundaries, pinpointing issues or optimizing specific parts of the circuit becomes challenging, if not infeasible. Alternatively, we can use the `--keep-hierarchy` flag when loading design with `read_slang`.

Student Task 4: Design hierarchy

- 4.1 Enable the *--keep-hierarchy* option of Slang and parse the design once again.
  - Generate the report and netlist again. Do you notice any difference with the previous version? What changed?
- Add the commands to read in the Croc design with Slang to the script template.

The rest of this exercise assumes you read the design with the *--keep-hierarchy* option enabled.

Design Elaboration

Elaboration transforms the HDL design into an intermediate representation, resolving the design hierarchy and instantiating all the modules. This step ensures that all components are in place before optimization can begin. The following commands are used during the elaboration stage:

```
yosys > yosys hierarchy -top <top_module>
yosys > yosys check
yosys > yosys proc
```

The `hierarchy` command is used to check, expand, and clean up the design hierarchy. In this step, the design is converted from a flat collection of module in Slang's AST representation format to an internal hierarchical graph representation with the top module as root node. The `-top <top_module>` flag is used to specify the top module for the design hierarchy. This top module should be the same one you specified during the design loading step. Any modules that are not part of this hierarchy will also be removed. The `check` command ensures that the design hierarchy is properly validated, generating an error if any unknown modules are used as cell types.

Note: `hierarchy` *command should always be the first command after the design is loaded. By specifying the top module, the* `(* top *)` *attribute is automatically added to it, which is required by other commands that rely on the top module for their operation.*

Once the design hierarchy is built, the next step is to translate the behavioral logic into a RTL netlist. This is done using the `proc` command. The `proc` command is a macro that invokes a series of sub-commands designed to convert high-level processes into multiplexers, flip-flops, and latches. Running this command transforms the design from a high-level description into a more optimized form, which is ready for further steps in the synthesis flow.

After this, the design is ready for coarse-grain synthesis. However, before proceeding further, it's a good practice to save the intermediate netlist and generate a report to review the synthesis progress.

Student Task 5: Elaboration

- 5.1 Build the design hierarchy and ensure the top module is correctly specified.
- 5.2 Translate processes to netlist with `proc` command.
- 5.3 Export the design and report
- Copy the commands you used to the script template.
- Check a simple synchronizer at `rtl/common_cells/sync.sv`. It become `croc_chip.i_croc_soc.i_croc.i_clint.i_sync` In the exported design. Observe what has changed.

Coarse-grain Synthesis

In Yosys, **coarse-grain synthesis** represents a stage where the design is still at a higher level of abstraction. At this stage, logic operations are expressed using **word-level operators**, which handle multi-bit data (such as 16-bit, 32-bit, or 64-bit words) instead of individual bits. These operators may include functional units like adders, multipliers, or shifters that process data chunks more abstractly. For example, a 32-bit adder is still identifiable as an adder, but its internal structure hasn't yet been broken down into individual gates like AND, OR, and NOT.

During this stage, Yosys handles important components like **finite state machines (FSMs)** and **memories**, converting them from higher-level behavioral descriptions into hardware-friendly representations. For instance, FSMs are encoded into state registers and control logic, while memories are mapped to technology-specific RAM blocks or synthesized using register arrays.

The order of transformations and optimizations at this stage plays a crucial role in ensuring optimal performance and resource utilization. For most of the steps described in the next section there exist a single command that performs all the transformations described. However, most of these commands are actually macros that expand to multiple commands when executed in the Yosys shell. Let us now go through the key steps and commands involved in coarse-grain synthesis.

First, the `check` command identifies potential design issues early in the synthesis flow. Running this command helps us catch errors before progressing further, saving time and preventing more complex failures later in the process.

FSM optimization (`fsm`)

Next, we will use the `fsm` command to extract and optimize Finite State Machines (FSMs) from the design. This step converts FSMs into hardware-friendly representations by optimizing state encoding, removing unreachable states, and simplifying transitions. You can use `yosys help fsm` to have more details, or refer to the dedicated documentation page (https://yosyshq.readthedocs.io/projects/yosys/en/latest/using_yosys/synthesis/fsm.html).

```
yosys > yosys fsm
```

Word Reduction (`wreduce`)

The `wreduce` pass reduces the bit-width of operations. For example, if a multiplication is specified with 16-bit inputs but only 8-bit precision is required, `wreduce` optimizes the operation by eliminating unnecessary bits, saving area and improving efficiency.

```
yosys > yosys wreduce
```

Peephole optimizations (`peepopt`)

The `peepopt` pass applies a collection of peephole optimizers to the current design. This includes architectural level optimizations such as simplifying arithmetic expressions. The command is:

```
yosys > yosys peepopt
```

Resource Sharing (`share`)

The `share` pass consolidates shareable resources into a single instance. It uses a SAT solver to determine if resources can be shared. This optimization is beneficial for reducing redundant resources, such as duplicated arithmetic units, in the synthesized netlist.

```
yosys > yosys share
```

Memory mapping (`memory`)

In HDL design, memory blocks such as SRAMs and ROMs can be either explicitly instantiated as macro blocks, or inferred by the tool from the behavioral description of the design. While for ASICs the explicit instatiation of previously-generated memory macros is the standard, memory block inference is mostly used on FPGA flows. The `memory` command is used to infer memory blocks from the design. In ASIC designs, this mainly serves to generate optimized address decoders and registers for large flip-flop arrays.

```
yosys > yosys memory
```

Note: For more information on these optimization passes, refer to the official documentation (https://yosyshq.readthedocs.io/projects/yosys/en/latest/using_yosys/synthesis/opt.html).

Design optimization

A critical aspect of improving synthesis quality is applying optimization passes. Optimization simplifies the design, reduces unnecessary logic, and enhances the circuit's efficiency. In this section, we will explore Yosys' optimization passes, which can be performed individually by means of specialized `opt_*` commands, or altogether with the `opt` macro.

Note: Optimization passes are essential because the initial synthesis output is rarely optimal. Often, it contains redundant logic, over-specified wire widths, or sub-optimal mappings that can be improved. Optimization helps clean up the design and makes it more suitable for implementation in hardware.

However, remember that the goal is not to create a perfect netlist but rather one that meets the design constraints efficiently. Avoid over-focusing on excessive optimizations, as it may lead to diminishing returns.

We can start by applying simple optimizations using the `opt` command. The `opt` pass calls several other `opt_*` passes in an ordered sequence, performing trivial optimizations and cleanups. Refer to the official documentation (https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/opt.html) for detailed information on the specific passes included.

The `opt` pass can be inserted before and/or after each major synthesis step, helping maintain a simplified design and potentially reducing runtime for the following steps. While it is generally useful to call `opt` frequently, doing so too often can increase synthesis time. As a rule of thumb, call `opt` when you expect tangible improvements.

Note: The `opt_dff` pass should be disabled before proper handling of Finite State Machines (FSMs). You can apply the optimization pass as follow:

```
yosys > yosys opt -noff
```

Additionally, the design may need further cleanup before moving to the next stages. You can achieve this by running the `clean` command, which tidies up the design. It is recommended to use this command after each major steps in the synthesis flow.

The final step in coarse-grain synthesis is optimizing the design's flip-flops using the `opt_dff` command. This pass optimizes away constant value flip-flops and merges reset and enable logic and synchronous reset multiplexers into them. This allows the tool to remove unused control signals, or even eliminate flip-flops entirely by replacing them with constant drivers where applicable.

Student Task 6: Coarse-grain synthesis

- 6.1 Perform an early check on the design. You should not encounter any errors.
- 6.2 Run a first set of optimizations on the design. Make sure to use the *-noff* flag to ensure that the `opt_dff` command is excluded from the optimization pass, as we have not optimized the design's FSMs yet.
- 6.3 Extract and optimize FSMs in the design with `fsm` command. Generate a report with the `stat` command and inspect it. Which changes do you notice with respect to the previous ones?
- 6.4 Perform bit-width reduction optimizations with the `wreduce` command. Perform arithmetic expression simplification with the `peepopt` command.
- 6.5 Clean the unused wires and cells with `opt_clean`. Optimize the design. Use the *-full* option of the `opt` command to enable further optimizations.
- 6.6 Perform resource sharing with `share`. Then run optimization with `opt` again.
- 6.7 Perform memory inference and register file optimization with the `memory` command. Run a optmization with *-fast* option.
- 6.8 Optimize the flip-flops in the design with `opt_dff` command. Use *-sat* option to remove flip-flops with inputs that can be proved to be constant by SAT solver. Use *-nodffe* and *-nosdff* to disable dff -> dffe with enable and diff -> sdff with synchronous reset/set transformations.
- 6.9 Run full optimization again. Clean the design with `clean`, use *-purge* to remove internal nets with public name.
- 6.10 Generate and analyze the report. What has changed after optimizing the flip-flops?
- Add the commands to the script template.
- Now, find `croc_chip.i_croc_soc.i_croc.i_clint.i_sync` again, observe what has changed.

Synthesis Constraints

In digital design, a set of specifications, such as **performance**, **area**, and **power**, must be met for a design to be valid. These specifications are referred to as constraints and must be communicated to the synthesis tool before the compilation process begins.

The constraints define your “wishes” to the synthesis tool. The tool will attempt to convert your HDL description into a gate-level netlist that meets these constraints. However, it’s important to note that while the tool will strive to satisfy these requirements, there is no guarantee that all constraints will be fully met. After synthesis, it is crucial to verify if the design meets the desired constraints, as the tool may not always be able to fulfill them due to resource limitations or conflicting specifications.

Yosys only partially supports constraints through the external ABC technology mapping tool. There are three key types of constraints you can define:

- **Clock Period**: This constraint specifies the maximum allowed clock period (or the minimum clock frequency) for the design. It defines how fast the design is intended to operate.
- **Driving Cell**: The driving cell constraint defines the type of cell used to drive the primary inputs of the design. This constraint allows to measure realistic input delays by modeling the input signals with a driving cell from the target technology library.

- **Output Load**: The output load constraint specifies the capacitive load that each output of the design will drive. It defines how much load each output will need to drive and helps the synthesis tool select appropriate output drivers.

Different timing paths

As depicted in Figure 2, the data paths of a sequential circuit can be subdivided into four categories with regard to their timings:

- *t_ss*: From state-holding registers to state-holding registers (s -> s),
- *t_is*: From input to state-holding registers (i -> s),
- *t_so*: From state-holding registers to output (s -> o),
- *t_io*: From input to output (i -> o).

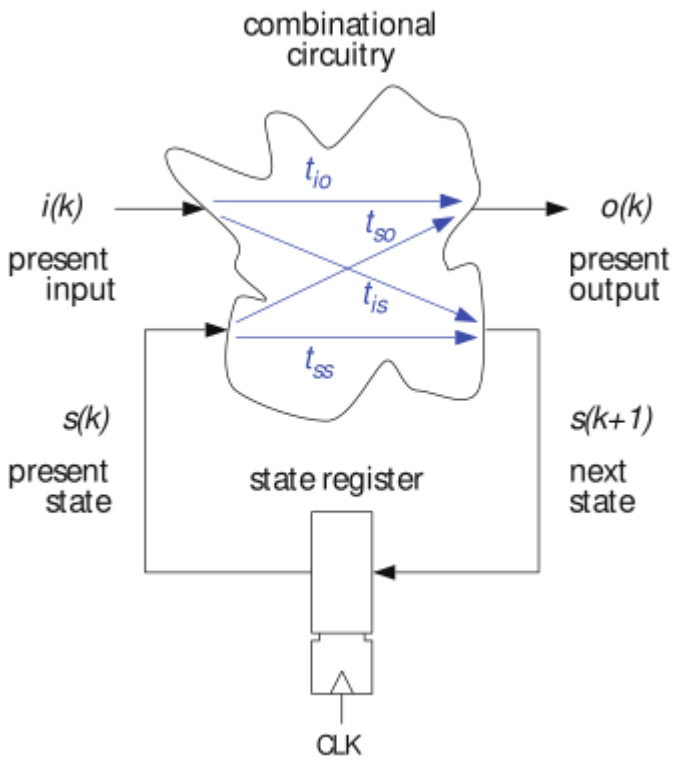

Figure 2: Signal propagation paths through single-phase edge-triggered synchronous circuits.

A meaningful starting point of a path is the input of our circuit (for *t_io* and *t_is*) or the clock input of a flip-flop (for *t_so* and *t_ss*). Similarly, the end points for a timing path are either circuit outputs (for *t_so* and *t_io*) or the input pin of a flip-flop (for *t_is* and *t_ss*).

Define a Clock Period

One of the most crucial constraints is the frequency at which the circuit is intended to operate. The clock period determines how fast the design can run, and thus it's essential to specify this constraint early in the design process. In this section, we'll set the target clock period for our sample design.

In the Yosys interactive shell, you can set up a design variable to define and trace parameters used throughout the whole synthesis flow. To set up a design variable, use the following command:

```
yosys > set <variable> <value>
```

The target frequency varies for each design, so you should establish performance goals based on your project's specific requirements. For our example design croc, previous implementations in the same technology node could achieve a frequency of 100 MHz. You can use this number as target frequency for this exercise as a sanity check that the reference flow you are building throughout the exercise is not close to optimal.

Student Task 7: Setup a Design Varable

- 7.1 Create a design variable for the clock period. In Yosys, the delay target (intended clock period) is specified in picoseconds by default (for more details, see Yosys ABC documentation (https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/abc.html)). To define a clock period of 10,000 picoseconds (which corresponds to 100 MHz), use the following command:

```
yosys > set period_ps 10000
```

Note that while we ask you to define a TCL variable at this point of the flow since we are now discussing constraints, there is no special reason for the variable to be called period_ps (nor a real need for a variable at all). Although for teaching purposed it is convenient to define all constraints at the same time, be aware that the only thing you actually need to do to specify timing constraints is to pass the clock period with a dedicated command line option to the `abc` command, as you will see in student task 11.

- Add this command to the corresponding section in the `yosys_flow.tcl` script.

Set Input Drivers and Output Load

In practice, input signals are not ideal, and their edges have finite rise and fall times due to the driving strength of the circuits providing these signals. The strength of the input driver affects the timing and overall performance of the design. In this section, we will define the driving cells for the input signals and set the output load that the design will drive.

The driving cell defines the strength of the input signal. By specifying a driving cell, you're telling the synthesis tool how strong or weak the input signals are, which will help the tool adjust the design accordingly. In our example design, we can use buffer cells (**sg13g2_buf_x**) as a reference input driving cells.

The numerical suffix `x` indicates the size and reflects the relative drive strength of the cell. A stronger buffer (e.g., sg13g2_buf_16) will drive loads faster. However, Using a buffer that's too strong might not accurately represent how the chip will perform, which might not mirror real-world scenarios. In our example design, set the driving cell as `sg13g2_buf_1`.

In the later stages of design, a constraint file is required to pass all the constraints to Yosys. We have provided a template for the ABC constraint file at `croc/yosys/src/yosys_abc.constr`. You can use the following command to specify the input driver cell:

```
> set_driving_cell <cell>
```

Propagation delay in CMOS circuits is critically influenced by the capacitive load that the output drives. A higher load capacitance means it takes longer for the output to charge or discharge, directly increasing the delay and affecting signal propagation speed. Using the buffer cell's input capacitance as a proxy for the load is a common practice because it reliably represents the effective capacitive load seen by the driving cell. Refer to the [standard cell documentation (https://github.com/IHP-GmbH/IHP-Open-PDK/blob/main/ihp-sg13g2/libs.ref/sg13g2_stdcell/doc/sg13g2_stdcell_typ_1p20V_25C.pdf)](https://github.com/IHP-GmbH/IHP-Open-PDK/blob/main/ihp-sg13g2/libs.ref/sg13g2_stdcell/doc/sg13g2_stdcell_typ_1p20V_25C.pdf), we can find for example, the input pin capacitance of `sg13g2_buf_8` is 0.0086pF, and `sg13g2_buf_16` is 0.0171pF.

In many cases, designers choose a higher capacitance value to ensure that timing constraints are met even under worst-case loading conditions.

You can use the following command to specify the output load:

```
> set_load <load>
```

Student Task 8: Set input / output constraints

- 8.1 Open the constraint file template and specify the input driver cell.
- 8.2 Open the constraint file template and specify the output load as 0.015pF. This roughly corresponds to the input capacitance of the `sg13g2_buf_16` gate. Note that the load value is expressed in picofarads (pF). At this point, the constraint file is complete and will be used in the subsequent technology mapping stage.

Technology Mapping

In the previous sections, we saw how HDL code is transformed into an RTL netlist. The RTL netlist is still described in terms of abstract, coarse-grain cell types such as wide adders, multipliers, and memory elements. Now, we will explore how this RTL netlist is transformed into a functionally-equivalent netlist composed of standard cell instances from the target technology library and wires connecting them, making it ready for physical implementation.

Technology mapping typically consists of two key phases:

- **Cell Substitution**: In this phase, the RTL cells are mapped to an internal Yosys library of generic (i.e. technology-independent) single-bit gate-level cells. This serves as an intermediate step before mapping to actual technology-specific cells.
- **Gate-Level Technology Mapping**: In this phase, the netlist consisting of single-bit gates is transformed into a netlist using gates from the target technology library. This step ensures that the design is composed of cells available in the specific fabrication process.

The goal of technology mapping is to ensure that the synthesized design can be implemented with the available technology, optimizing for factors such as timing and area.

Cell Substitution

Cell substitution is performed using the `techmap` pass. This step replaces RTL cells with simpler, lower-level cells, either from a provided Verilog file or Yosys's built-in mappings. By default, if no specific map file is provided, the techmap pass uses a built-in map file to substitute Yosys RTL cell types with internal gate-level cells.

Student Task 9: Generic Cell Substitution

- 9.1 Run the `techmap` pass to replace RTL cells in the design with Yosys's internal gate-level cells. Run the `opt` and `clean` after the maping.
- 9.2 Generate and analyze the report.
  - By looking at the report, can you tell what changed?
- Now, find `croc_chip.i_croc_soc.i_croc.i_clint.i_sync` again, observe what has changed.

Gate-level Technology Mapping

At the gate level, the target technology is described by a *Liberty* files (the **.lib** format), which contain information about available cells and their timing and power consumption characteristics. Liberty files are imported at the beginning of the synthesis flow, as explained in task 2. Technology mapping at this stage is performed in two key phases:

First, the **register cells** are mapped to the registers available in the target technology using the `dfflibmap` pass. This pass expects the Liberty file as an argument (using the -liberty option) and only uses the register cells from that file. You can use the following command for register cell mapping:

```
yosys > yosys dfflibmap -liberty <lib>
```

Second, the **combinational logic** is mapped to the target technology cells. Yosys relies on the external [ABC (https://github.com/berkeley-abc/abc)](https://github.com/berkeley-abc/abc) tool (via the `abc` command) for this step. ABC also takes the Liberty file as input but focuses on the combinatorial cells. Additionally, ABC allows you to specify constraints, such as the desired clock period (delay target), driving cell and output load. We use the following command for the combinational logic mapping:

```
yosys > yosys abc -liberty <lib> -D <period_ps> -constr <constr_file> [-script <abc_script>]
```

For the sake of brevity in this exercise, we provide you with a custom ABC optimization script `croc/yosys/scripts/abc-opt.script` that automates an iterative optimization and mapping flow. The script first loads the design and applies multiple rounds of high-effort delay optimization—utilizing structural hashing (`&sh`), rewriting (`&if`), synthesis (`&syn2`), and balancing (`&b`)—to refine the circuit's logic structure. It then interleaves optimization iterations (`&opt_iter`) with mapping iterations (`&map_iter`) that select and map the logic to a target technology, while continuously updating design statistics (`&ps`). Finally, the script performs topological sorting (`topo`), timing analysis (`stime`), and physical adjustments such as buffering (`buffer`) and cell resizing to further optimize delay and fanout, resulting in a well-optimized, delay-aware netlist. For your own project, you may want to develop an optimization script suitable for your design. Refer to the [ABC documentation (https://people.eecs.berkeley.edu/~alanmi/abc/)](https://people.eecs.berkeley.edu/~alanmi/abc/) for more information.

Since the design has been mapped to the technology libraries, we can now specify the liberty files using the `-liberty` flag to generate a more accurate report.

```
yosys > yosys stat -liberty <file>
```

Student Task 10: Technology Mapping

- 10.1 Perform the `dfflibmap` pass to map registers from the target technology. Select the stdcell library to make sure only registers from the stdcell library is used.
- 10.2 Generate and analyze the report. what changes have been made? Again, find `croc_chip.i_croc_soc.i_croc.i_clint.i_sync`, observe what has changed.
- 10.3 Next, you will map the combinational logic using the `abc` command.
  - Select the same stdcell Liberty libraries used in previous stage.
  - Pass the intended clock period via the design variable `period_ps` defined in the previous task. Note that TCL variables need to be referenced with the `${my-var}` syntax.
  - Define constraints by passing the `src/yosys_abc.constr` constraint file to the `abc` command.
  - Specify the provided ABC optimization script through the *-script* option.

  - Note that this step can take some time, depending on the amount of optimizations specified in the ABC script. You should expect it to run for 5-10 minutes in this exercise.
  - Add a `clean` with *-purge* command to clean up the design after abc command.

- Export the optimized netlist to a file (e.g. `out/croc.v`).

Static Timing Analysis

Yosys currently lacks an integrated static timing analysis tool; therefore, we will use [OpenSTA (https://github.com/The-OpenROAD-Project/OpenSTA)](https://github.com/The-OpenROAD-Project/OpenSTA) for design timing analysis. A simple example script is provided at `croc/sta/scripts/opensta.tcl`.

Student Task 11: Timing analysis

- Change the netlist name in the provided example script with the path to the last netlist exported by Yosys (it should be in the `yosys/out` directory).
- Start a new terminal and use the following command in the shell to execute the timing analysis and generated timing report:

```
sh (yosys) > cd ../sta
sh (sta)   > oseda -2025.01 sta scripts/opensta.tcl
```

- Generate the report and export the netlist.
  - What is the estimated design area?
  - What is the slack of the design?
  - What is the maximum clock frequency achieved?
- Does the design meet the intended frequency?

Our design currently does not achieve the target speed of 100MHz. Note that to simplify the inspection of the different reports we retained the hierarchy of the original design. However, retaining hierarchy prevents cross-boundary optimizations between modules and, depending on the design, it can be very detrimental in terms of achieved performance. In the next task you will learn how to flatten the design in Yosys while retaining the hierarchy of some critical parts of your design.

Flatten the Design

In our current synthesis flow, we successfully transform the HDL design into a gate-level, technology-dependent netlist. However, our design still fails to meet the timing requirement even after employing the customized ABC script. One potential solution is to use design flattening. Additionally, an important observation is that the `abc` pass **consumes a significant amount of time**. This is because the `abc` pass is applied separately to each module in the design hierarchy, leading to extended runtimes for designs with many hierarchical levels or multiple modules.

Flattening the design can address this inefficiency. When the design is flattened, all submodules are merged into a single module, allowing ABC to optimize across the entire design at once. This gives ABC better visibility over the full logic, enabling optimizations that cross module boundaries and ultimately results in a faster runtime and more efficient synthesis.

To flatten the design, add the following command in your script **before** technology mapping:

```
yosys > yosys flatten
```

Student Task 12: Flatten the design

- 12.1 Re-run the synthesis flow (you can leverage the script you filled in so far). This time, flatten the design before performing technology mapping. Add a `clean` with *-purge* option to clean the design after flattening.

- Export the netlist once again and generate the report.
  - What is the estimated design area?
  - What is the slack of the design?
  - What is the maximum clock frequency achieved?

- Does the design meet the intended frequency now?

- Inspect the generated netlist, what might be the disadvantages of flattening?

Note: Flattening offers clear benefits in runtime and synthesis efficiency. However, it also introduces certain limitations:

- **Reduced Design Clarity**: The original hierarchical structure often represents functional blocks. Flattening removes this structure, making the design harder to understand.
- **Debugging Challenges**: A fully flattened design loses its modular structure, making it more difficult to trace signals or debug issues.
- **Tool Limitations**: Some back-end tools rely on hierarchy information to apply constraints (e.g., timing or power), so fully flattening could disrupt later stages of the flow where this information is needed.

To balance the benefits of flattening with the need for clarity and debugging, you can use the `keep_hierarchy` flag. This flag allows you to preserve parts of the design hierarchy while flattening the rest, enabling optimizations without sacrificing modularity in **critical blocks**. Before executing the `flatten` command, you can select specific design elements to preserve their hierarchy:

```
% > yosys setattr -set keep_hierarchy 1   <module_name>
```

In your own project, you may want to keep the design hierarchy for modules that are critical for debugging or that encapsulate complex functionality. For example, if a module handles complex interfaces or timing-critical operations, preserving its hierarchy can make it easier to isolate and resolve issues. Conversely, simpler or less critical sections can be flattened to maximize optimization. In Croc, example of modules that would benefit from retained hierarchy visibility include important parts of the core (e.g. the register file), peripherals, and system-level configuration registers (see `soc_ctrl_reg_top`). It is also good practice to retain hierarchy over special modules such as Clock Domain Crossings (CDCs), synchronizers, and tech cells wrappers (`tc_clk*`, `tc_sram`), to avoid unwanted optimizations that would be difficult to spot later on. An example of module where you definitely do not want to fully retain hierarchy is the system's crossbar, since it contains long combinational paths across multiple modules that highly benefit from cross-module optimizations.

Student Task 13: Selective flattening

- Apply the `keep_hierarchy` flag to the chosen modules and re-run the synthesis flow.
- Regenerate the synthesis report. Record the key results:
  - What is the estimated design area?
  - What is the slack of the design?
  - What is the maximum clock frequency achieved?
- Does the design meet the intended frequency?

Prepare for OpenROAD Flow

This is the last step of the synthesis flow! So far, we have successfully transformed the HDL code into a gate-level netlist and mapped it to the target technology node. Now, we will prepare the netlist for the physical design step using OpenROAD. The following commands are used to align the output netlist with OpenROAD's requirements:

- `setundef` - Replace undefined (`x`) constants with defined (`0/1`) constants.
- `splitnets` - Split multi-bit nets into single-bit nets.
- `hilomap` - Map constants to 'tielo' and 'tiehi' driver cells.

Splitting Multi-bit Nets: Multi-bit nets represent groups of wires that carry multiple bits of data (e.g., buses). While multi-bit nets are convenient for high-level descriptions, physical design tools like OpenROAD expect netlists to operate at a more granular, wire-level representation (i.e., single-bit nets). By splitting the multi-bit nets into single-bit nets, we make the netlist more compatible with such tools, ensuring that each wire can be routed and optimized individually during the place-and-route phase. This also avoids ambiguity during physical design when dealing with buses.

Replacing Undefined Constants: In digital design, undefined constants (`x` states) represent an unknown or indeterminate value. While these undefined values might be used in simulation or verification to catch potential bugs, physical design tools like OpenROAD require deterministic values (either `0` or `1`) for implementation. Undefined values cannot be physically mapped to transistors, so by replacing them with defined constants, we ensure the netlist is physically realizable and that there are no ambiguities that could lead to synthesis or routing issues.

Student Task 14: Prepare for OpenROAD

- 14.1 Split multi-bit nets into single-bit nets using the `splitnets` command.
- 14.2 Set undefined ports to zero value using the `setundef` command. Do a `clean` afterward.

- 14.3 Replace constant `hi` and `lo` bits with corresponding driver cells from the technology library.
- Finally, export the netlist. The netlist is ready for further processing in the OpenROAD toolchain.

Congratulations! You have done with this Exercise.

Discuss your experience with assistants and colleagues or continue to explore Yosys and the provided design on your own.

Cleanup

Student Task - Final Cleanup

Only for ETH users:

- If you followed the instruction at this point you have installed the exercise files in a directory on the workstation's local hard disk (e.g. somewhere in `/scratch/$USER`). By default there is no limit to the amount of local disk space that you can use. Please note that the Tardis workstations are shared among several courses and have limited disk space. Therefore, please make sure to cleanup after yourself once the exercise is done. This is particularly important in the case of exercises that generate large files (e.g. long simulation logs, large netlists, and other files generated by the backend tools).

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).